BUKTI KERJA - SCREENSHOT & OUTPUT CODING

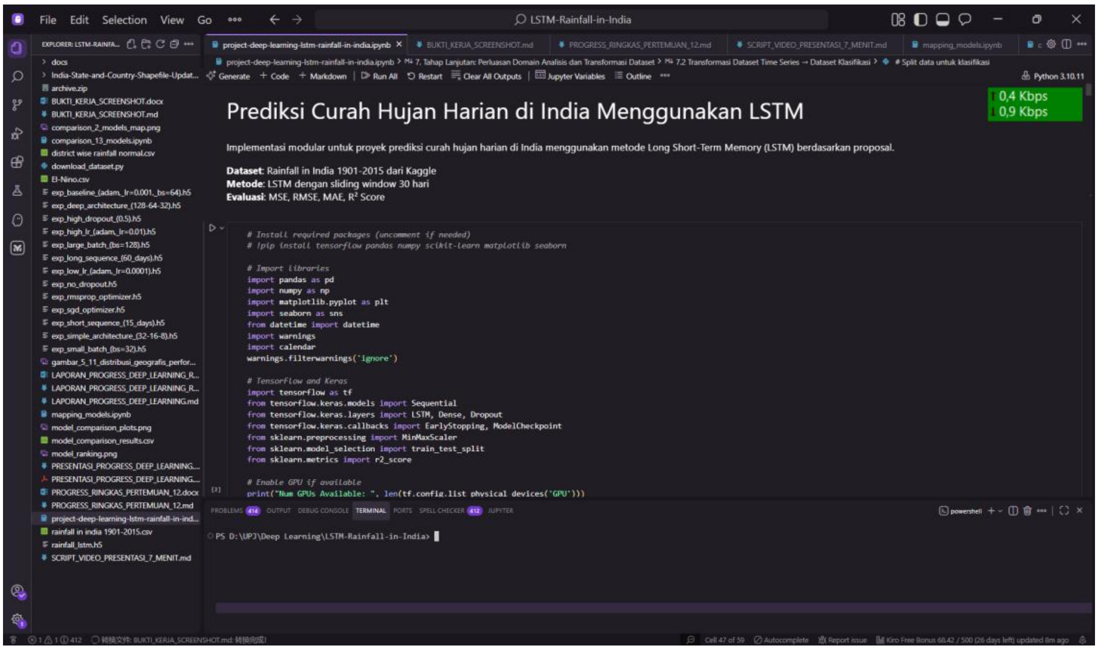
Prediksi Curah Hujan Harian di India Menggunakan Long Short-Term Memory (LSTM): Pendekatan Multi-Domain untuk Mitigasi Bencana, Perencanaan Pertanian, dan Manajemen Sumber Daya Air

1. SCREENSHOT NOTEBOOK & ENVIRONMENT

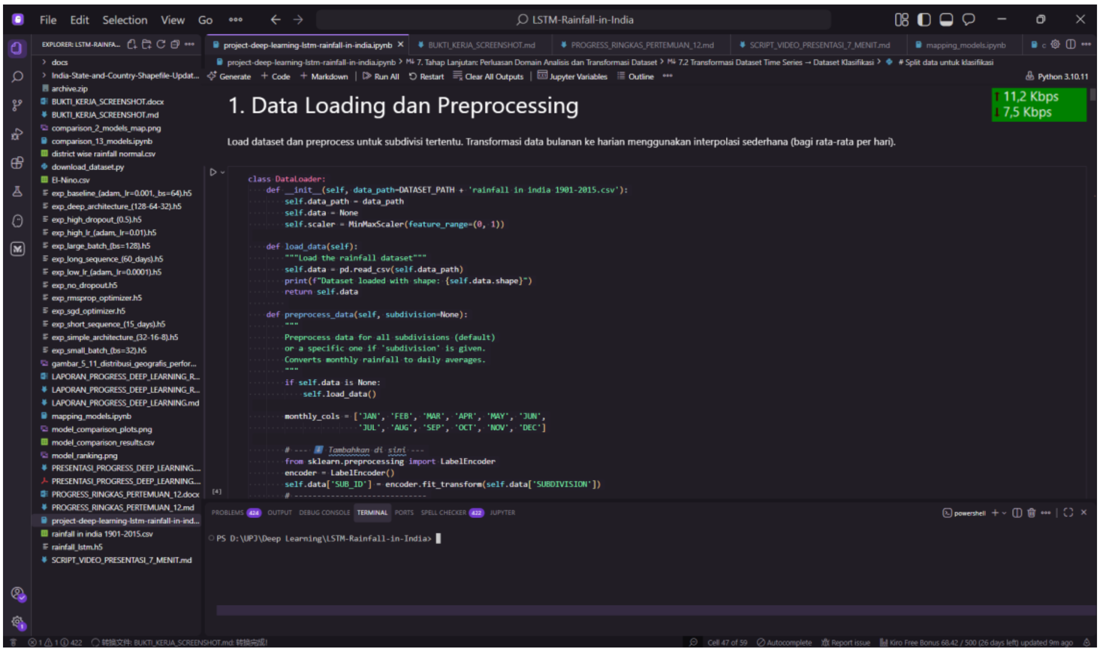
1.1 Jupyter Notebook - Main Project

File:project-deep-learning-lstm-rainfall-in-india.ipynb

Screenshot 1: Import Libraries & Setup



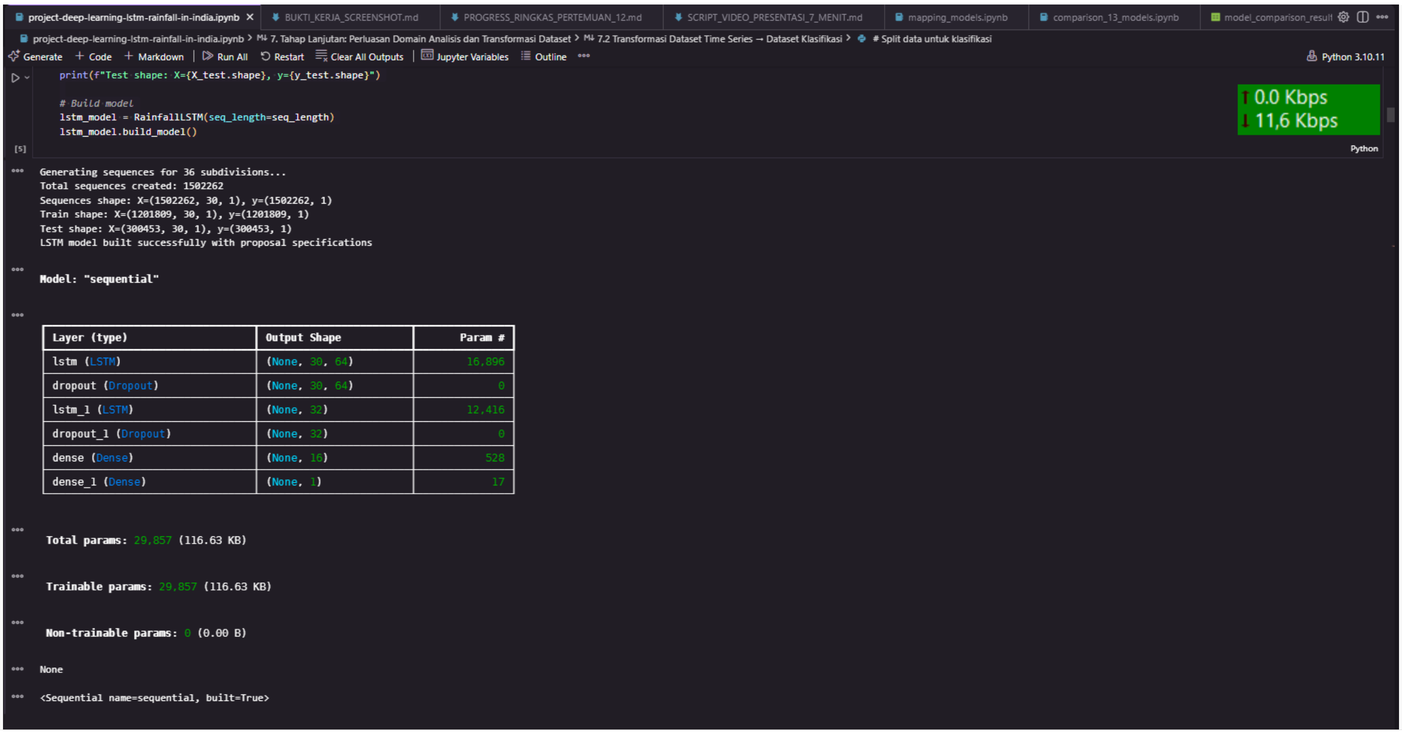
Screenshot 2: Data Loading & Preprocessing



2. ARSITEKTUR MODEL & TRAINING

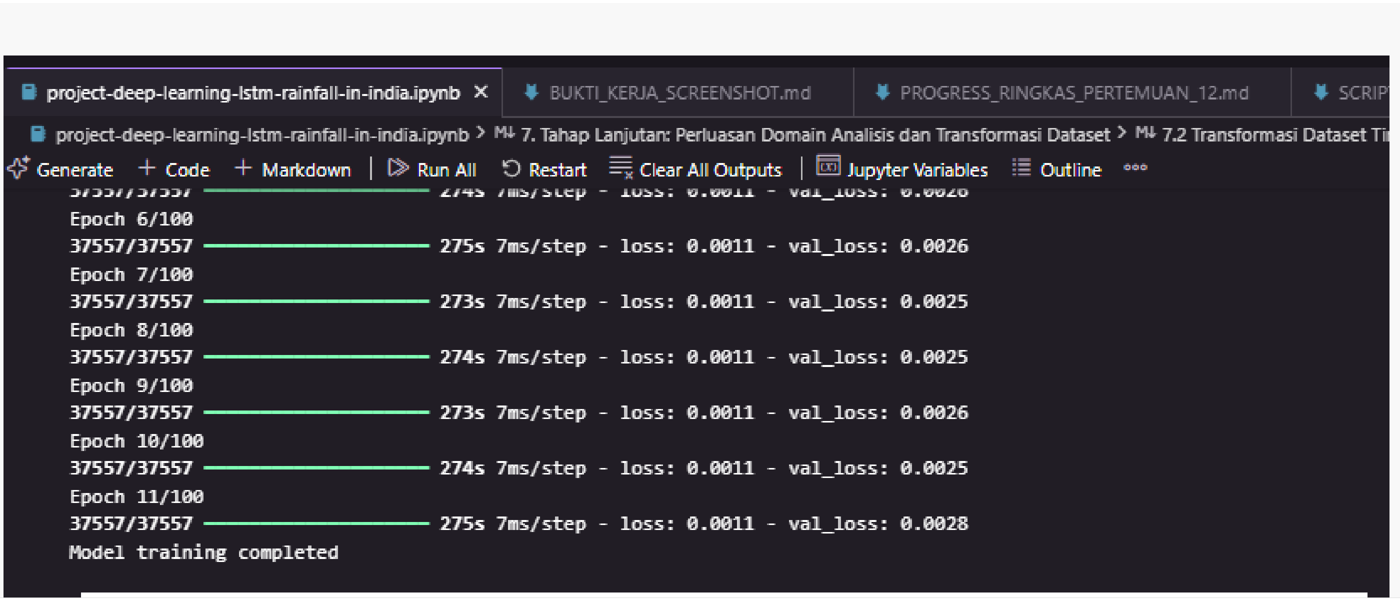
2.1 Model Architecture

Screenshot 4: LSTM Model Summary



2.2 Training Process

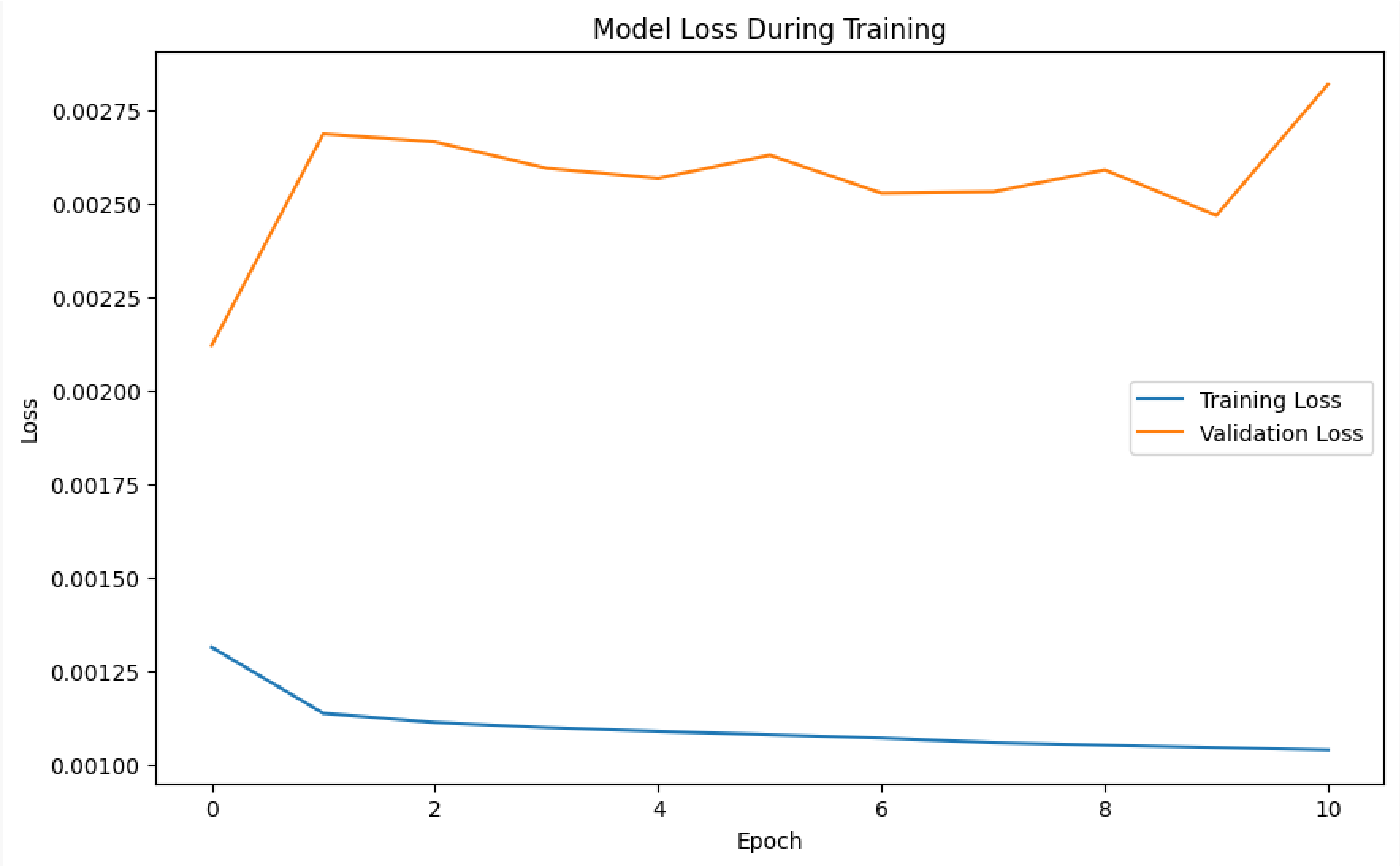
Screenshot 5: Training Progress



3. GRAFIK HASIL TRAINING

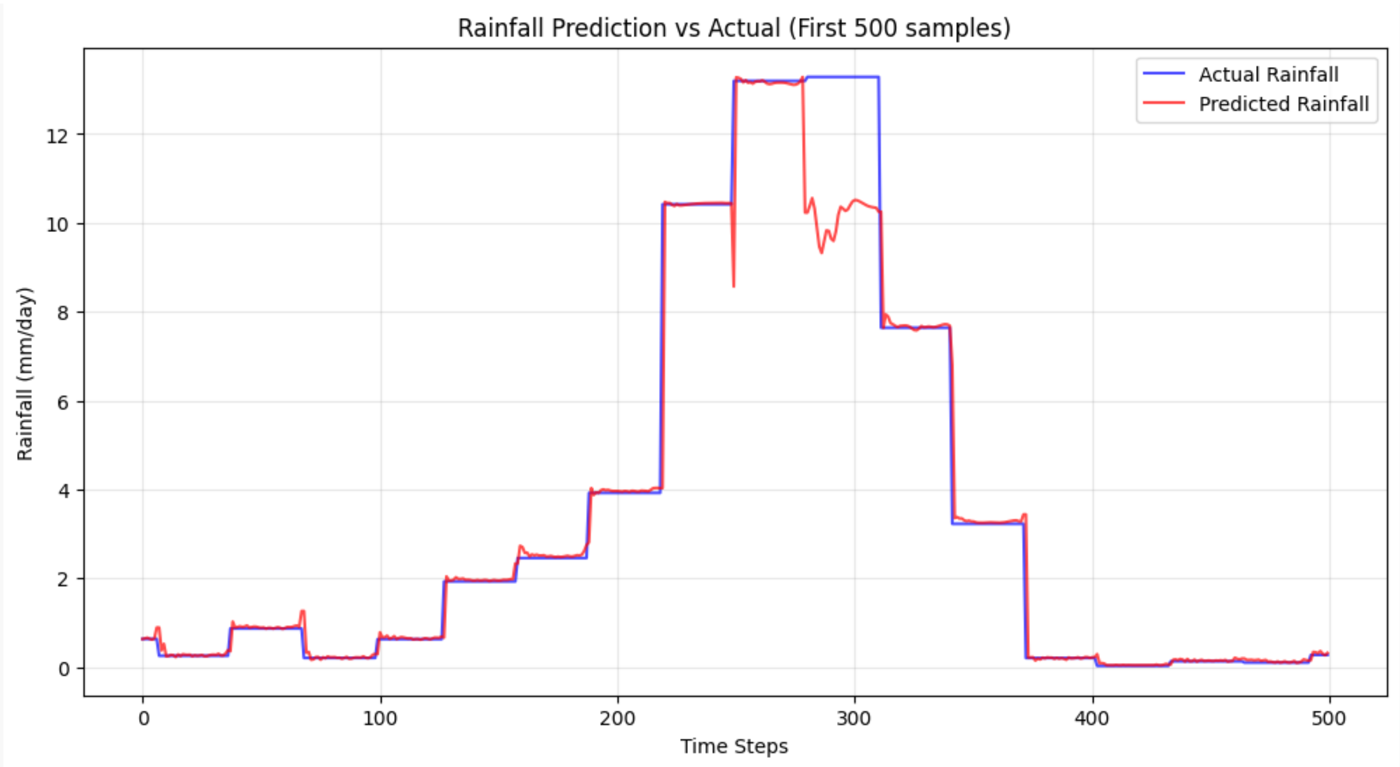
3.1 Training & Validation Loss

Grafik 1: Loss Curves

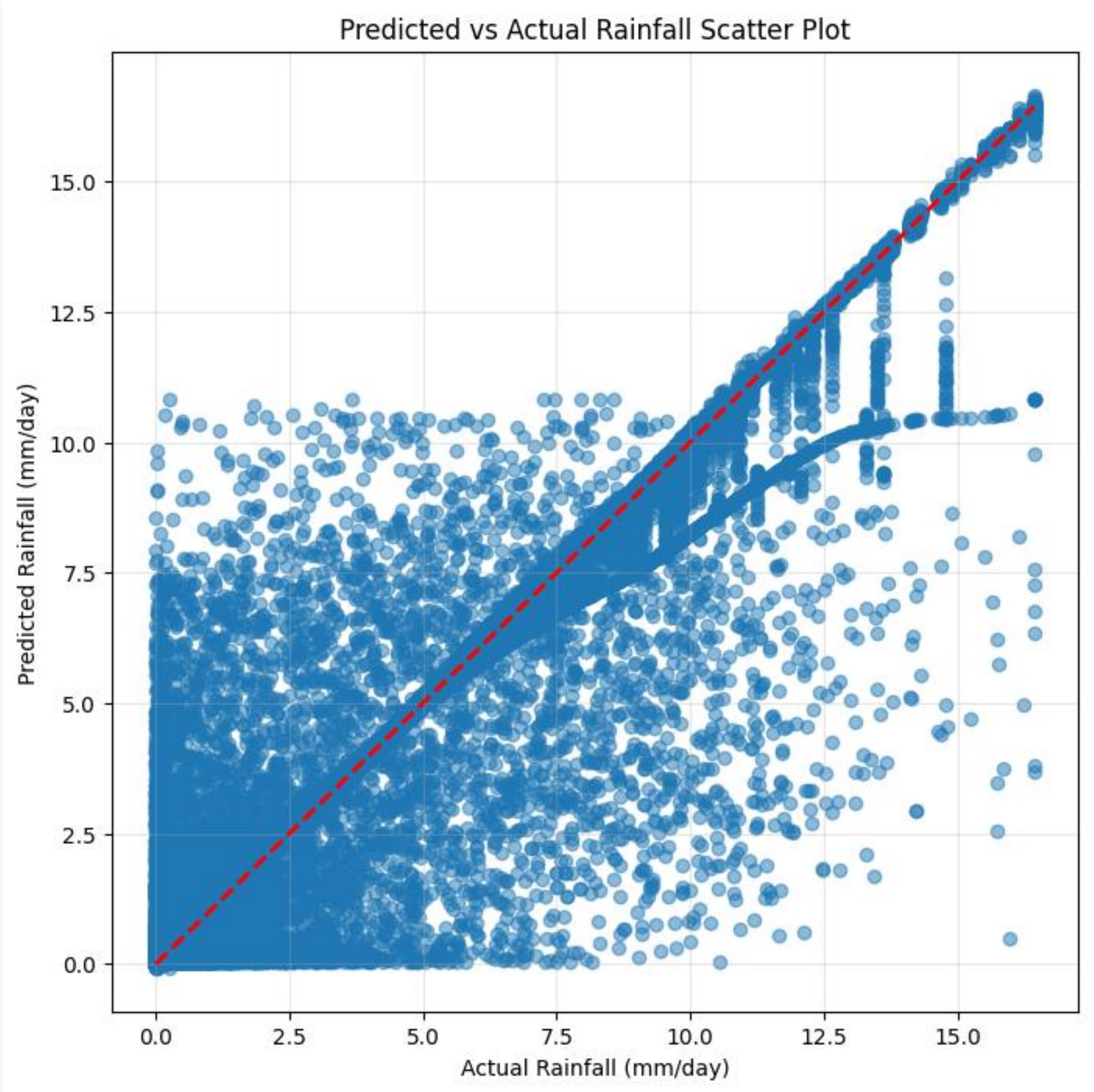


3.2 Prediction Results

Grafik 2: Predicted vs Actual (Time Series)



Grafik 3: Scatter Plot (Predicted vs Actual)



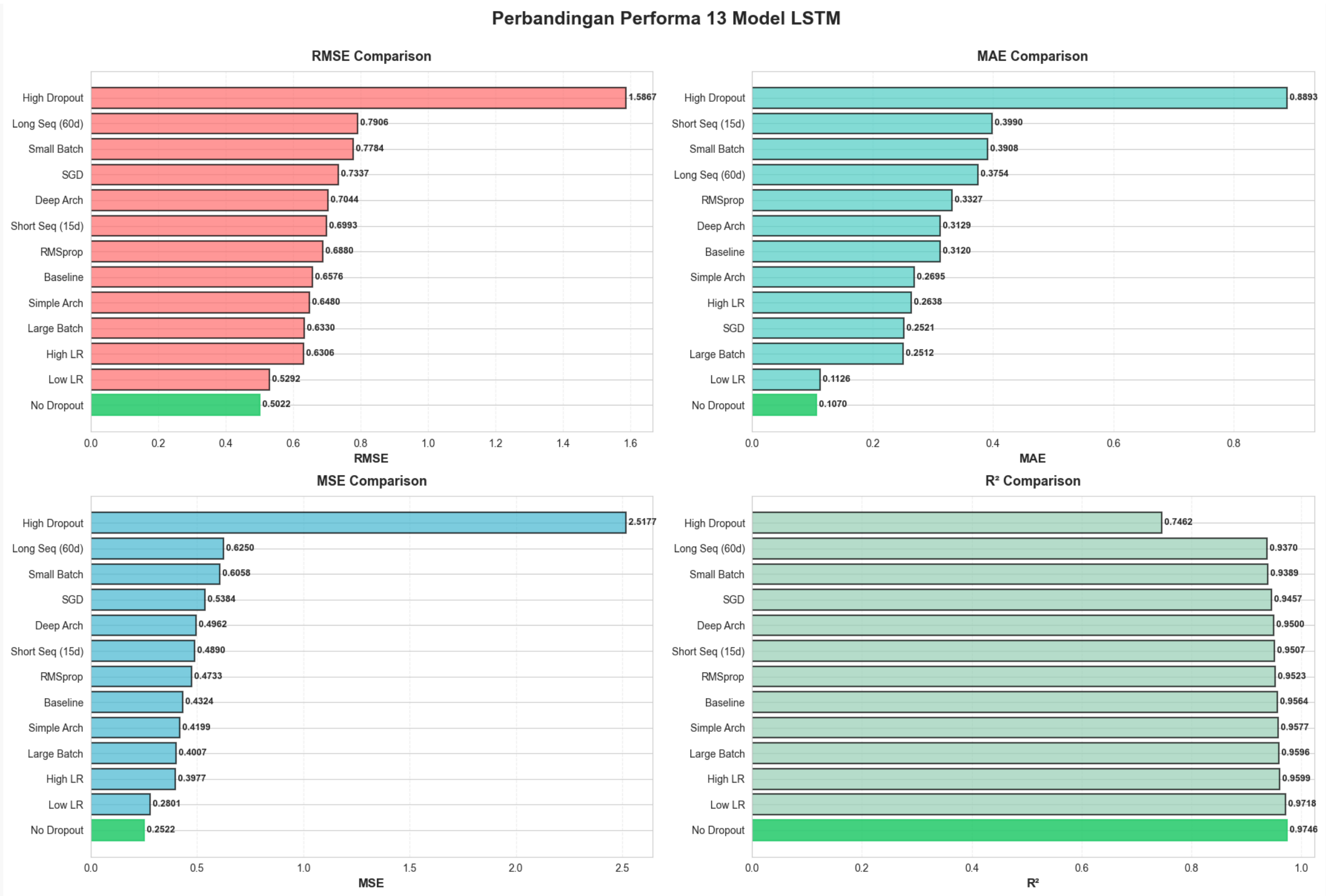
4. EKSPERIMEN 13 MODEL

4.1 Comparison Results

Model Comparison Table

Peringkat	Model	RMSE (mm)	R² Score
1	No Dropout	**0.5022**	**0.9746**
2	Low Learning Rate (0.0001)	0.5292	0.9718
3	High Learning Rate (0.01)	0.6306	0.9599
4	High Batch Size(64)	0.6330	0.9596
5	Simple Architecture	0.6480	0.9577

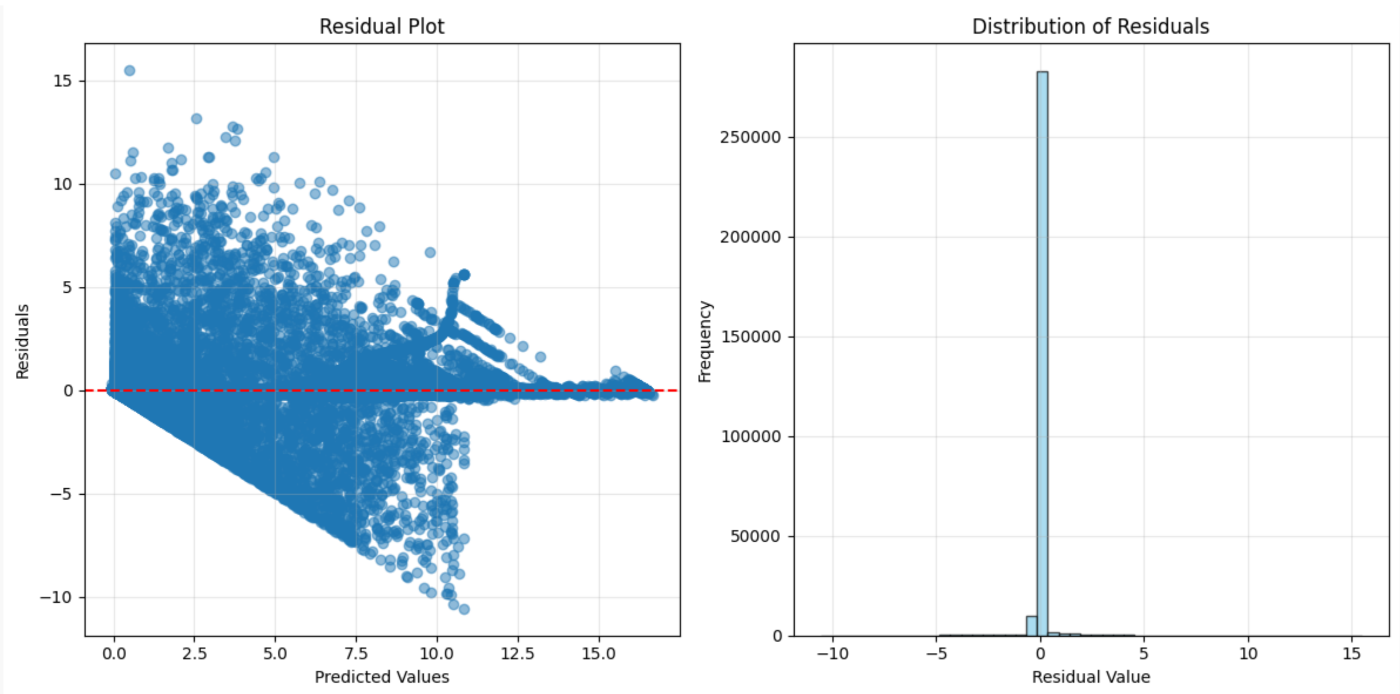
Grafik 4: Bar Chart Comparison (RMSE)



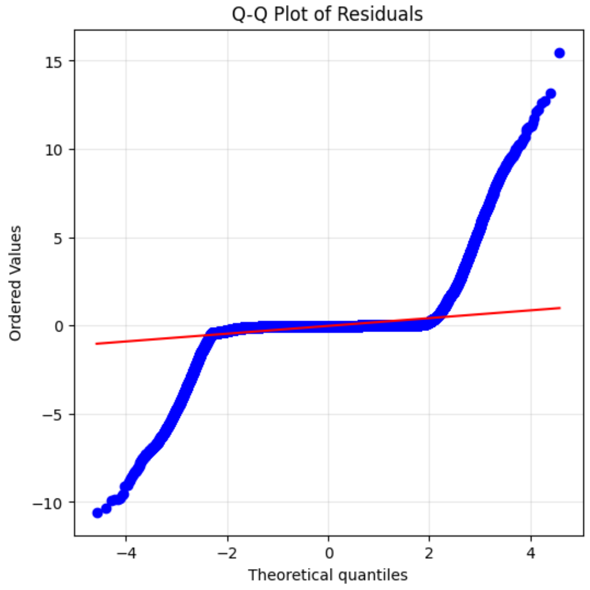
5. ANALISIS RESIDUAL

5.1 Residual Distribution

Grafik 5: Histogram Residuals

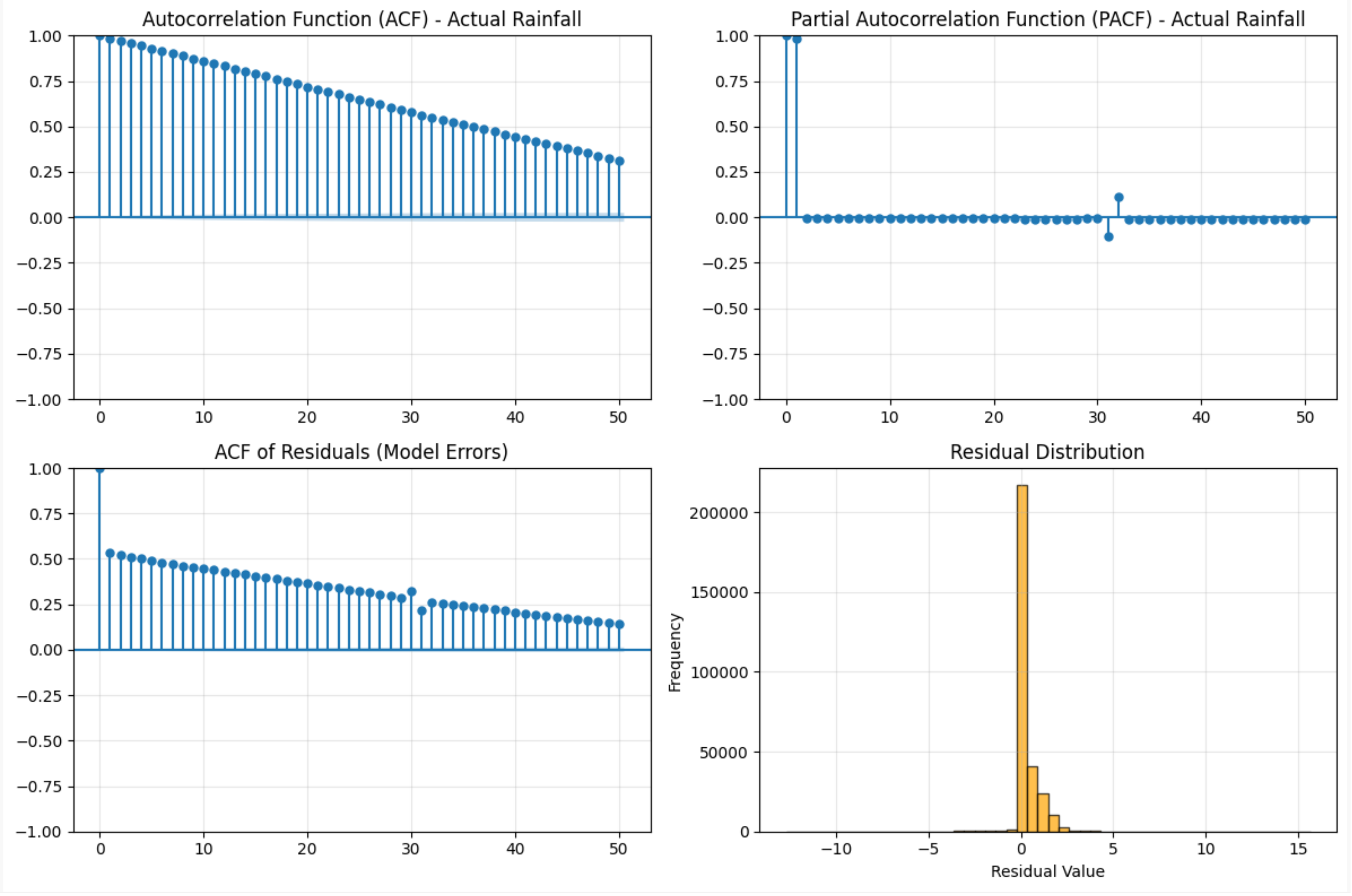


Grafik 6: Q-Q Plot



5.2 Autocorrelation Analysis

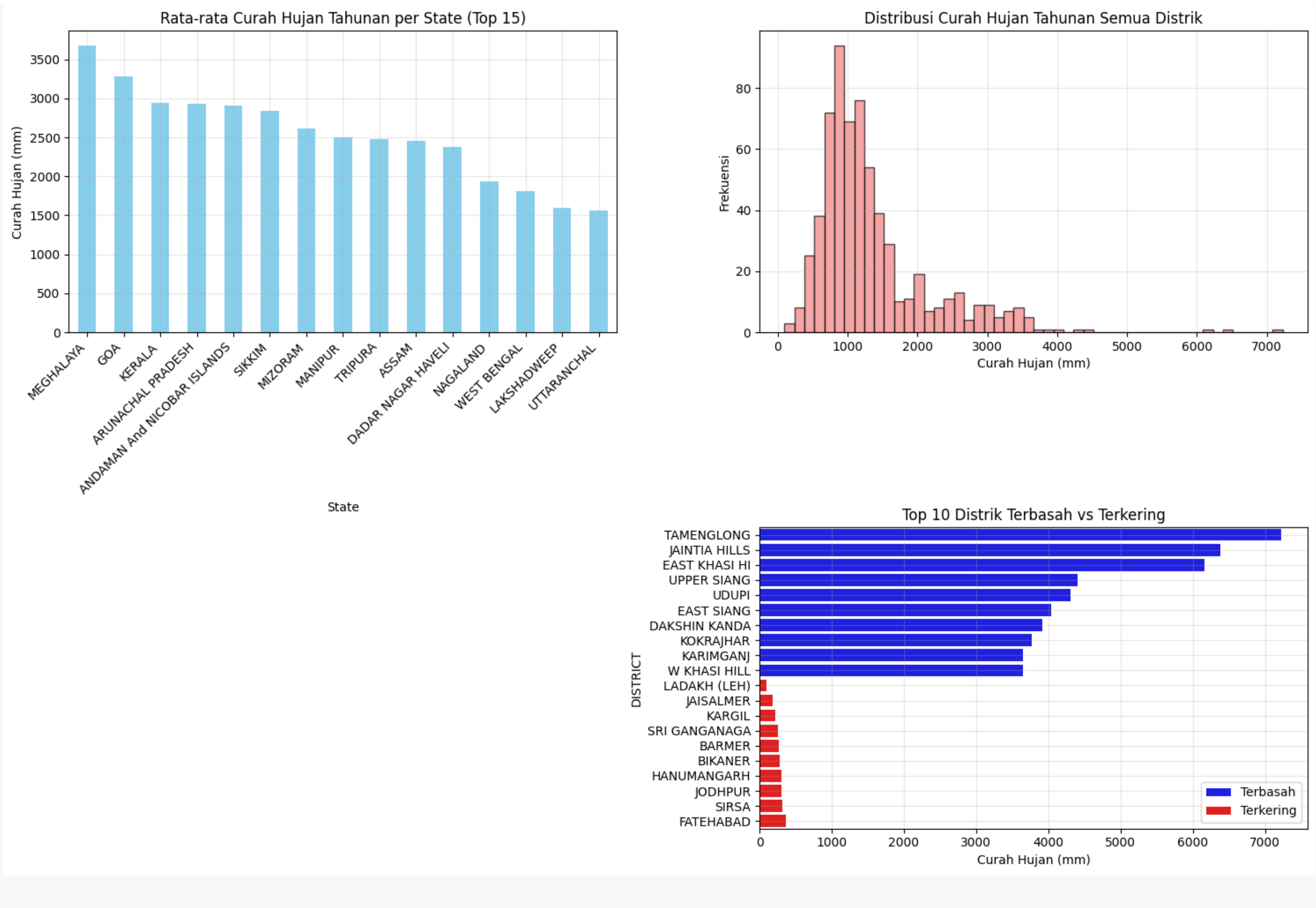
Grafik 7: ACF Plot



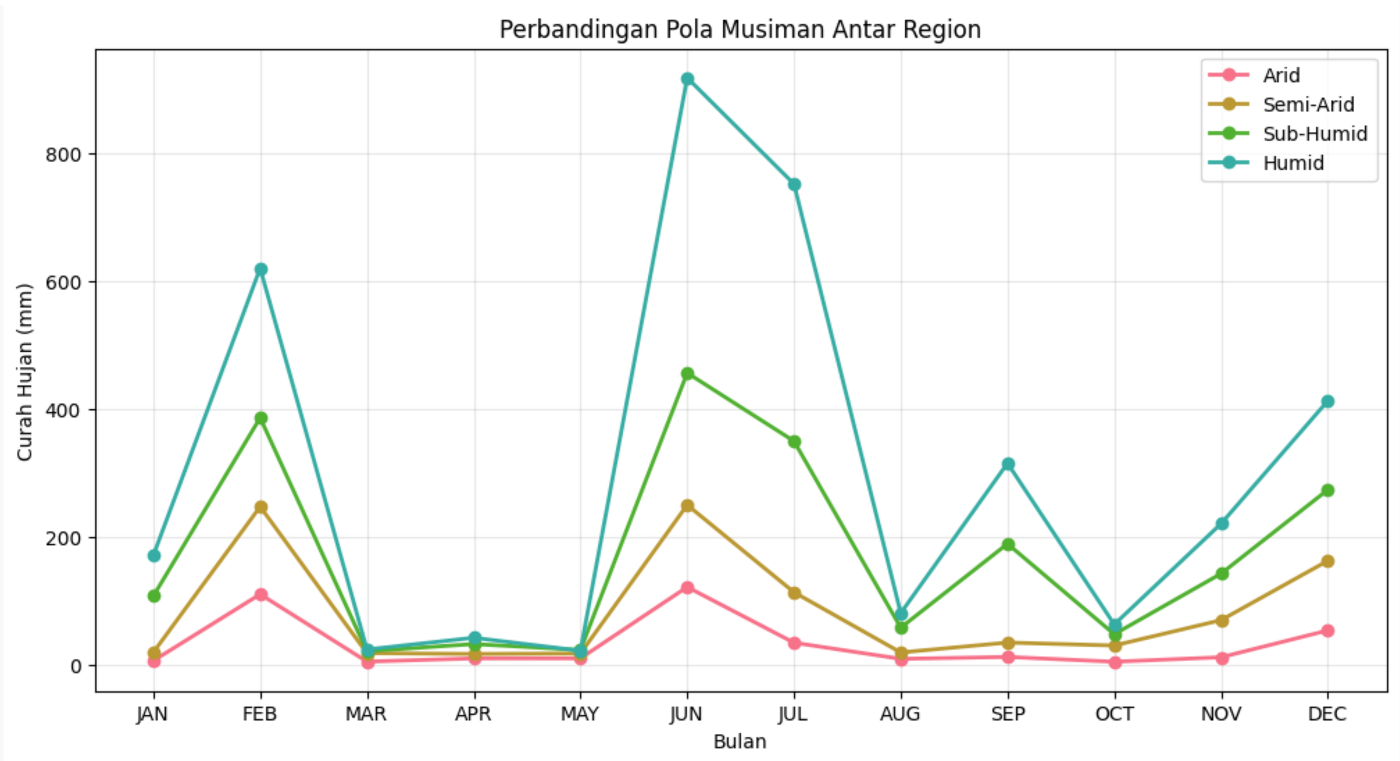
6. VISUALISASI DATASET

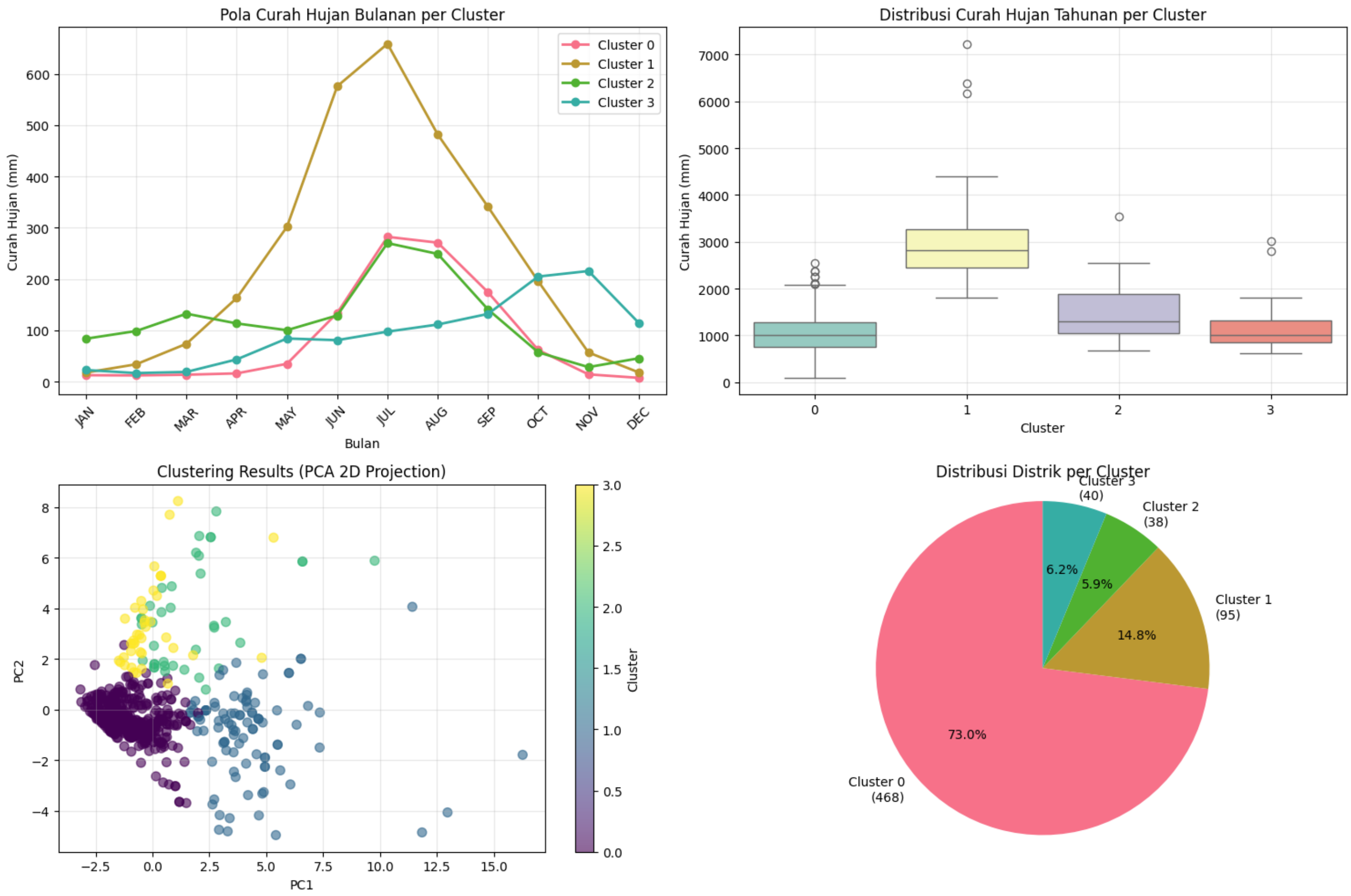
6.1 Data Distribution

Grafik 8: Distribusi Curah Hujan Tahunan



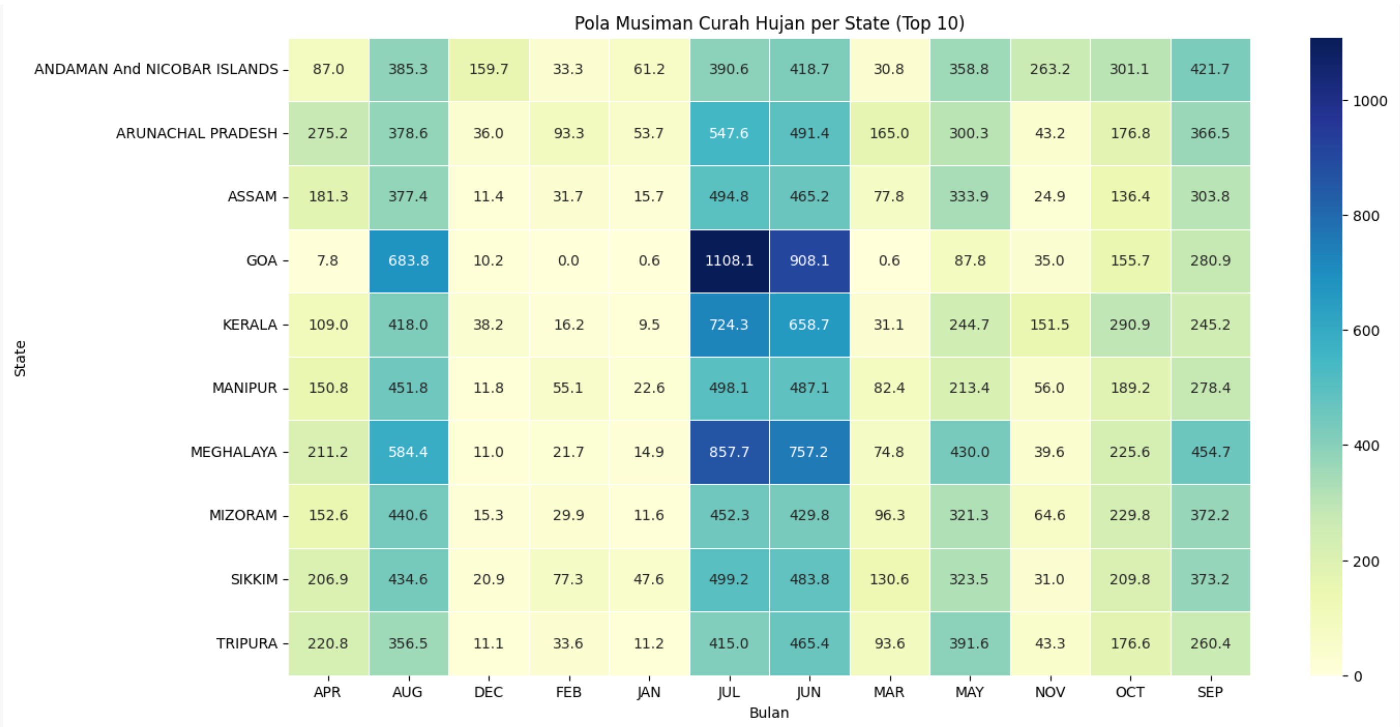
Grafik 9: Pola Musiman





6.2 Geographic Visualization

Grafik 10: Heatmap Curah Hujan per State



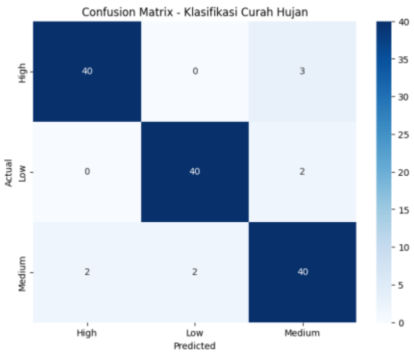
7. ANALISIS LANJUTAN

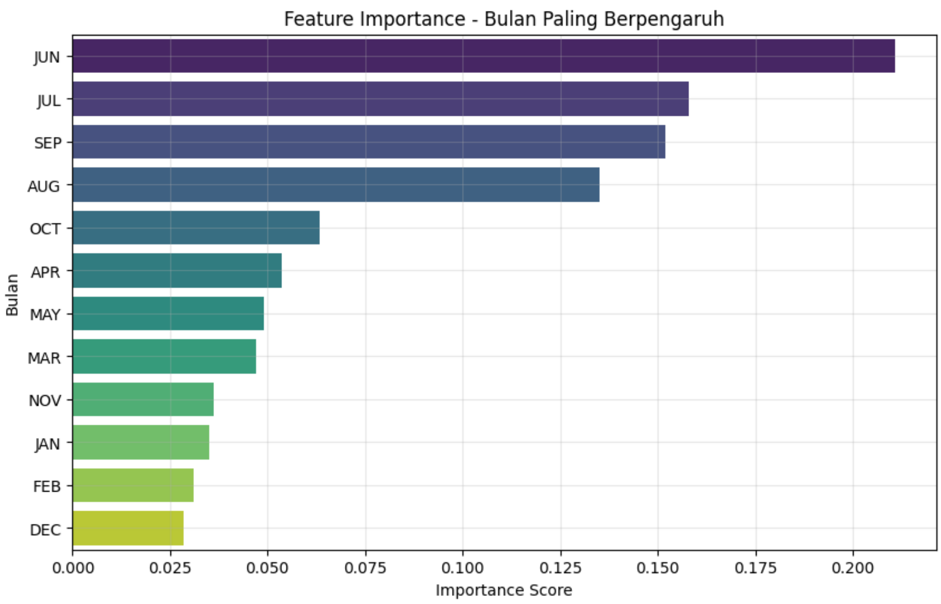
7.1 Klasifikasi (3 Kategori)

Screenshot 7: Classification Report

```
Data split untuk klasifikasi:
Train: 512 samples
Test: 129 samples

Training Random Forest Classifier...

=== HASIL KLASIFIKASI CURAH HUJAN ===
Classification Report:
              precision    recall  f1-score   support

        High       0.95      0.93      0.94        43
         Low       0.95      0.95      0.95        42
      Medium       0.89      0.91      0.90        44

    accuracy                           0.93       129
   macro avg       0.93      0.93      0.93       129
weighted avg       0.93      0.93      0.93       129
```

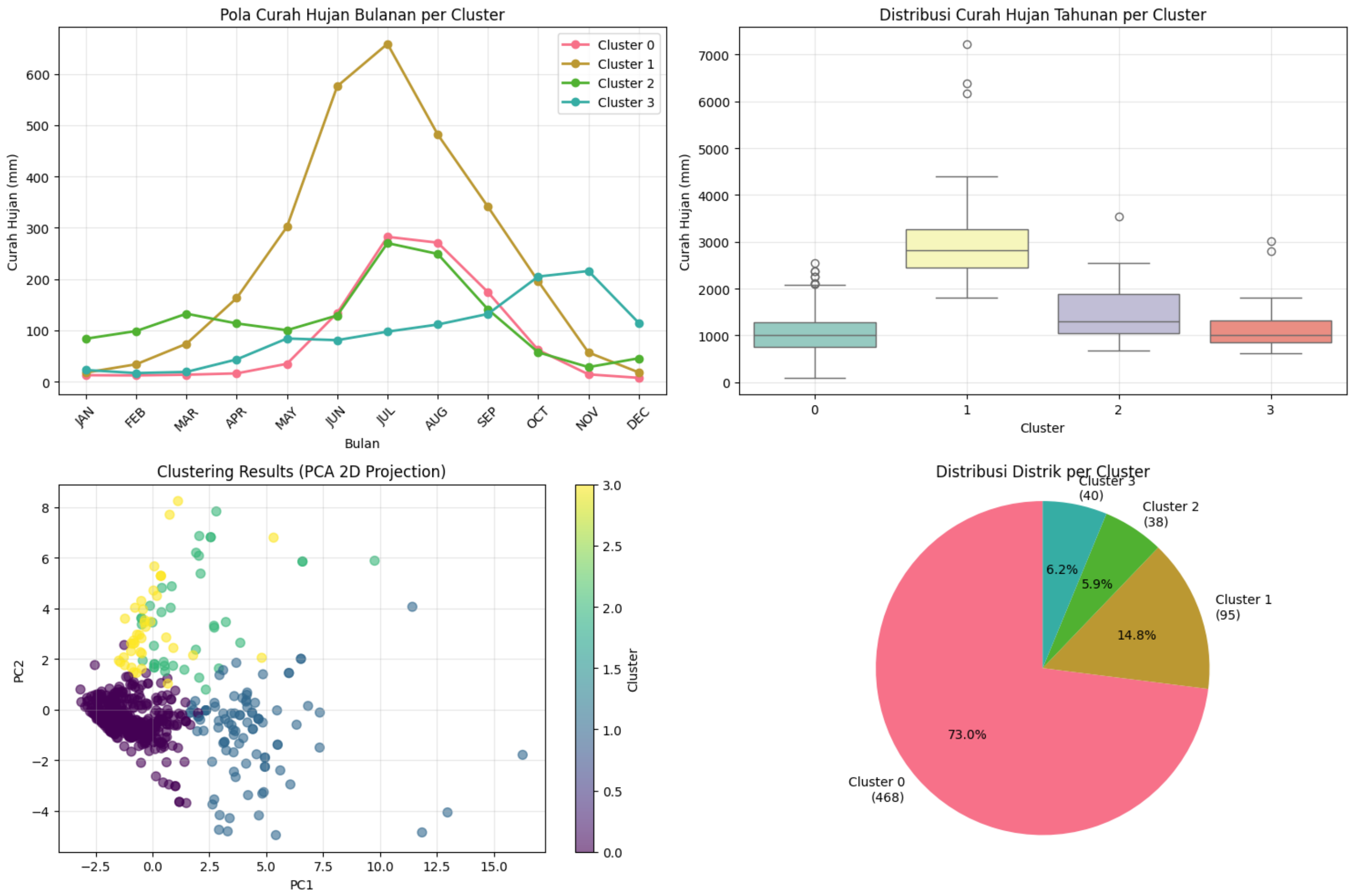
Grafik 11: Confusion Matrix



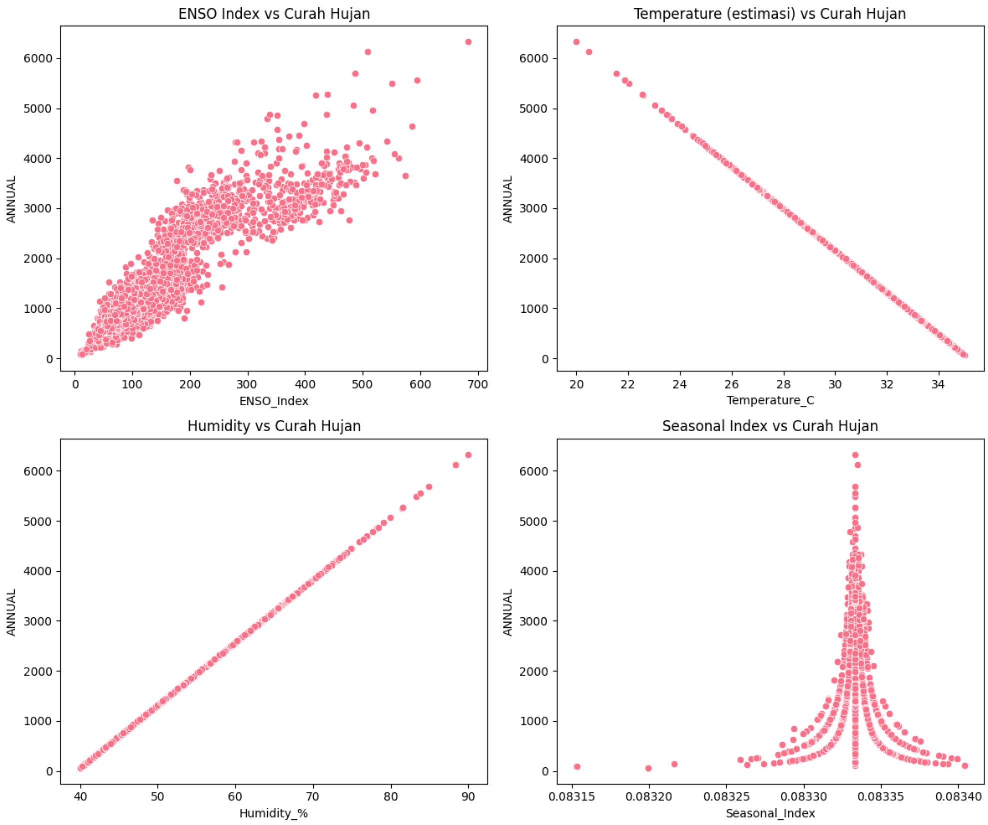


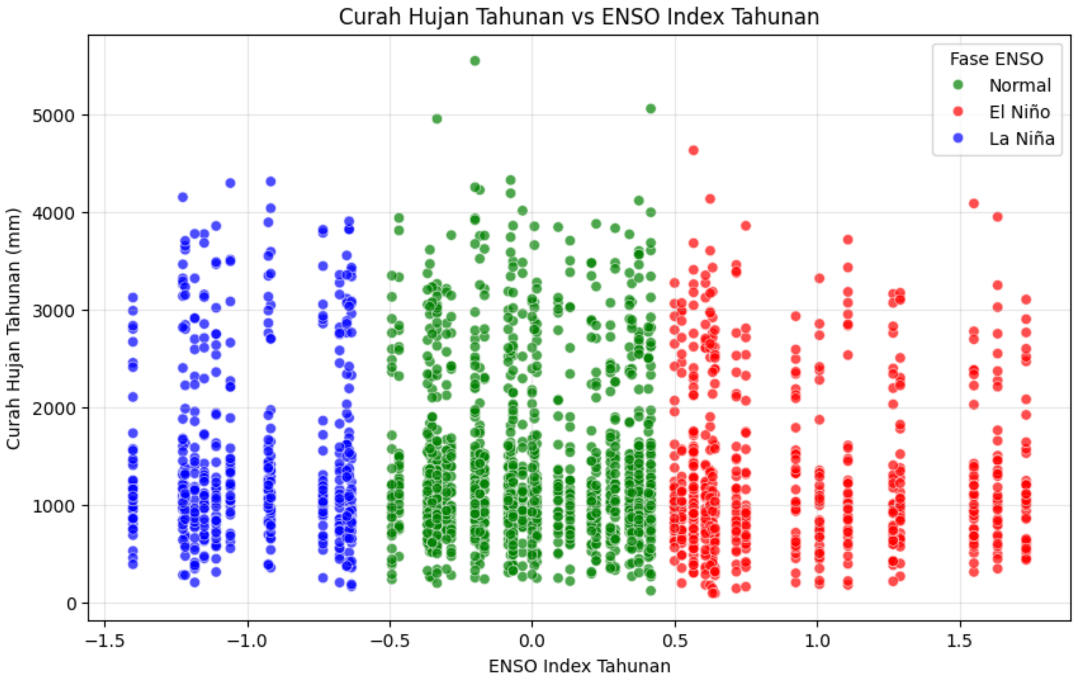
7.2 Clustering Analysis

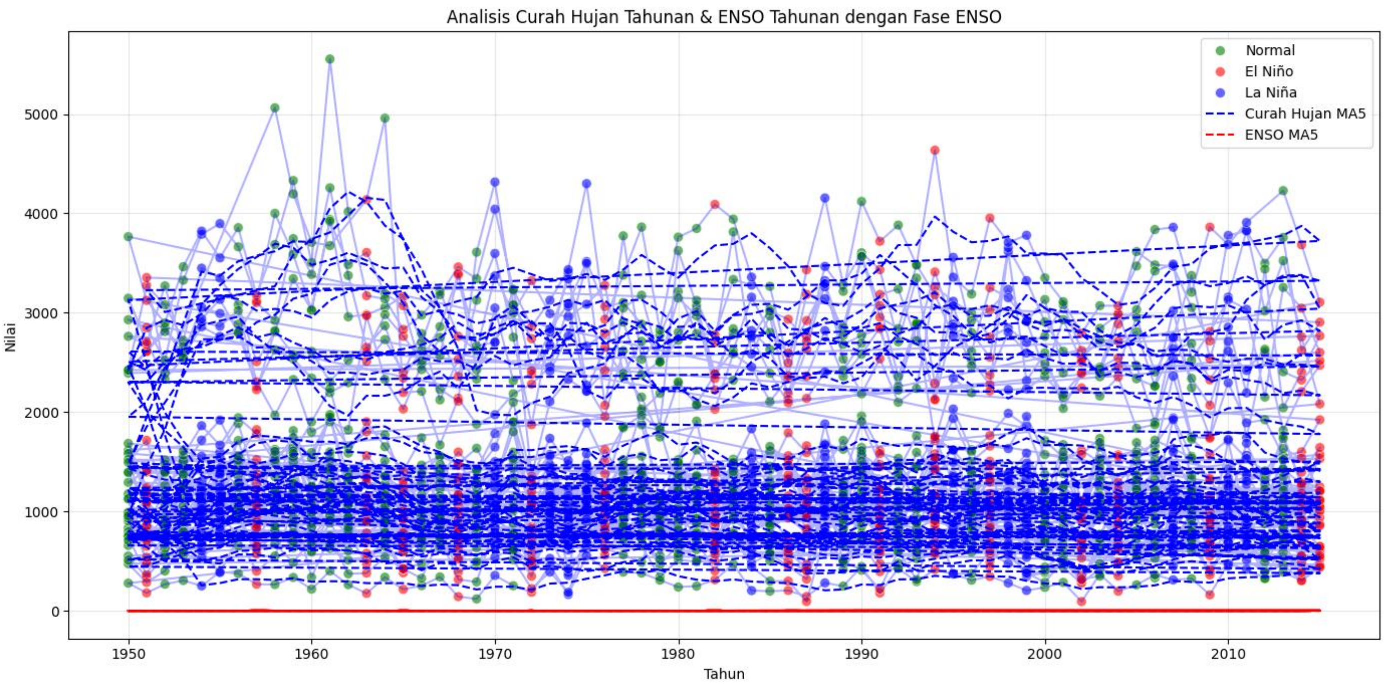
Grafik 12: K-Means Clustering (PCA 2D)



7.3 Perluasan Domain Analisis

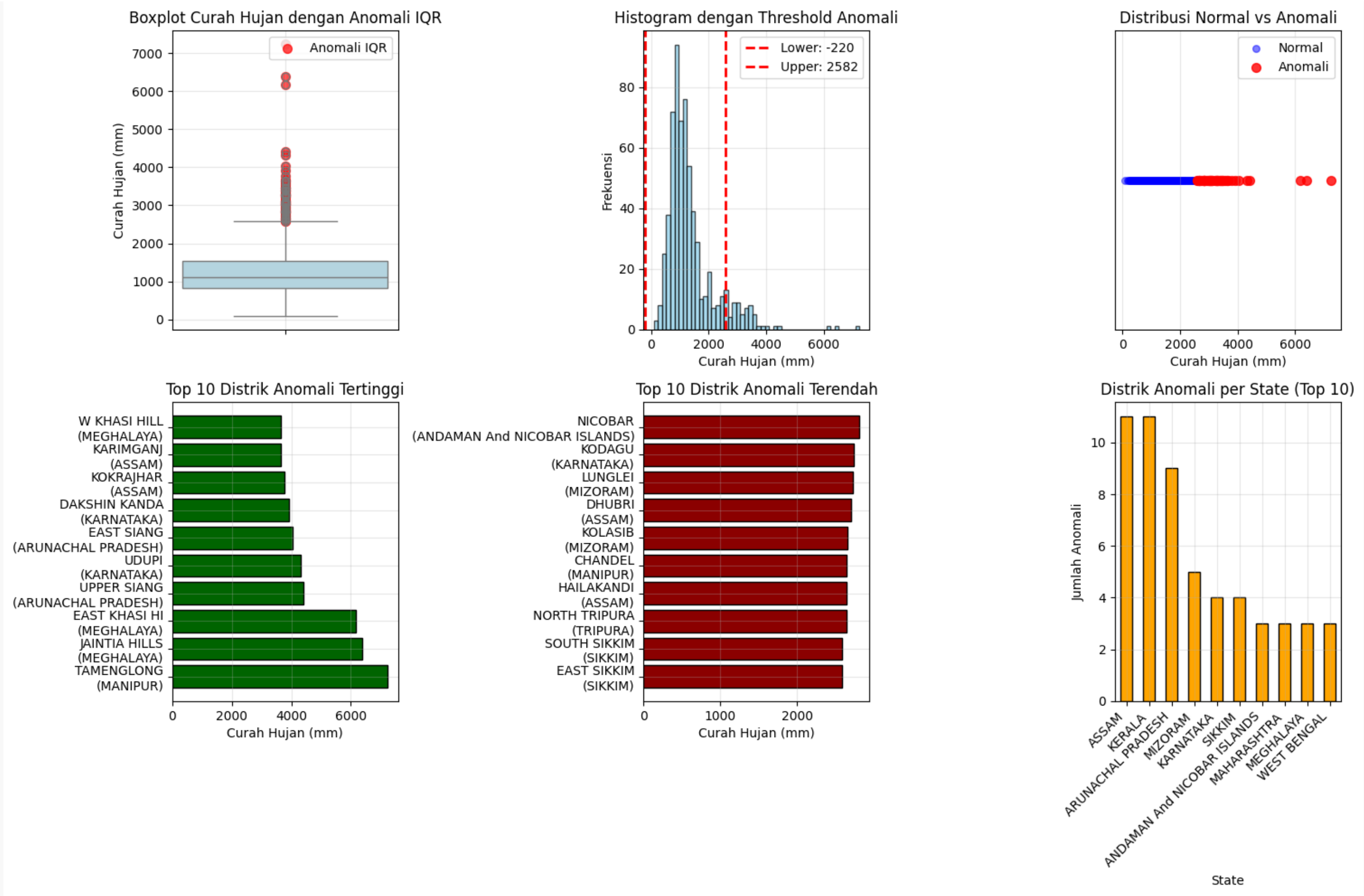



Curah Hujan Tahunan vs ENSO Index Tahunan
Fase ENSO
Normal
El Niño
La Niña
Curah Hujan Tahunan (mm)
ENSO Index Tahunan
0
1000
2000
3000
4000
5000
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5


Analisis Curah Hujan Tahunan & ENSO Tahunan dengan Fase ENSO
Normal
El Niño
La Niña
Curah Hujan MA5
ENSO MA5
Nilai
Tahun
0
1000
2000
3000
4000
5000
1950
1960
1970
1980
1990
2000
2010

7.4 Anomaly Detection

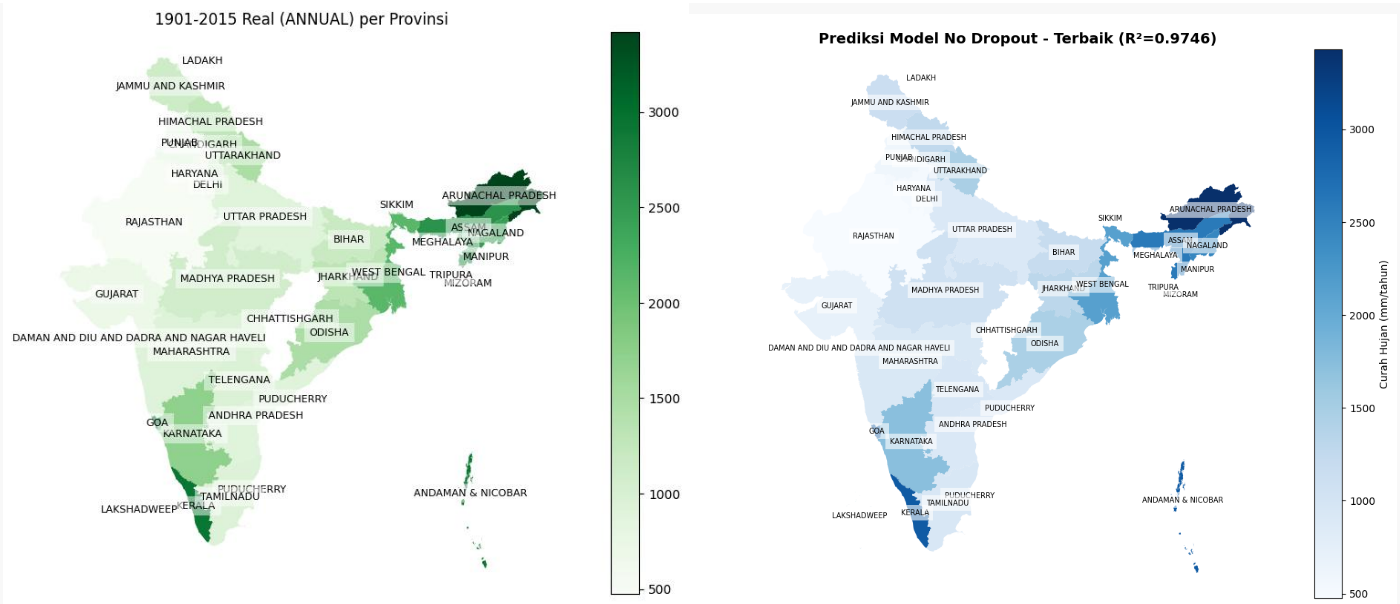
Grafik 13: Boxplot Deteksi Anomali



8. MAPPING MODELS (PERBANDINGAN GEOGRAFIS)

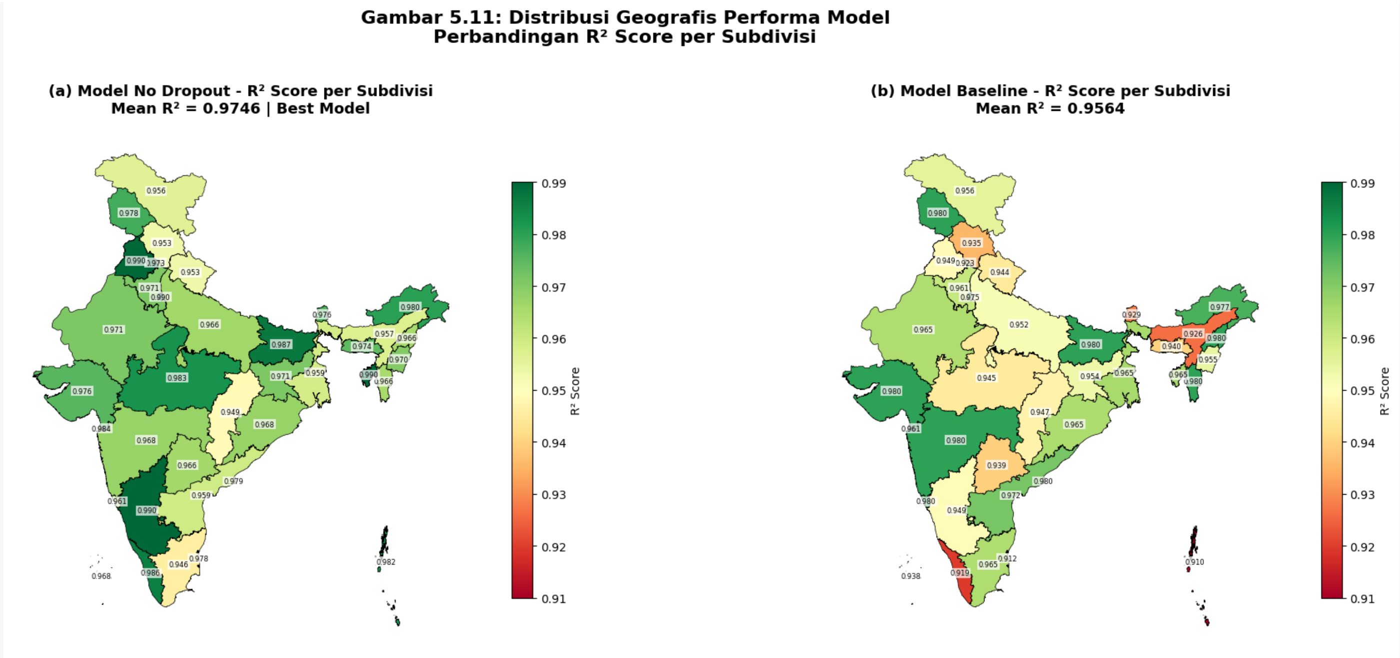
8.1 Peta India - Data Real vs Prediksi

Screenshot 8: Dual Choropleth Maps



8.2 Performa Geografis (R^2 Score)

Grafik 14: Distribusi R^2 per Subdivisi



9. POTONGAN KODE PENTING

9.1 Data Preprocessing

Code Snippet 1: Monthly to Daily Transformation

python

```python
def monthly_to_daily(df):
    daily_data = []
    for year in df['YEAR'].unique():
        year_data = df[df['YEAR'] == year]
        for month in ['JAN','FEB','MAR','APR','MAY','JUN',
                      'JUL','AUG','SEP','OCT','NOV','DEC']:
            rainfall = year_data[month].values[0]
            days_in_month = calendar.monthrange(year, month_num)[1]
            daily_rainfall = rainfall / days_in_month
            for day in range(days_in_month):
                daily_data.append(daily_rainfall)
    return np.array(daily_data)
```

9.2 Sequence Creation

Code Snippet 2: Create LSTM Sequences

python

```python
def create_sequences(data, seq_len=30):
    X, y = [], []
```

```
    scaler = MinMaxScaler()
    scaled_data = scaler.fit_transform(data.reshape(-1,1))

    for i in range(len(scaled_data) - seq_len):
        X.append(scaled_data[i:i+seq_len])
        y.append(scaled_data[i+seq_len])

    return np.array(X), np.array(y), scaler
```

9.3 Model Building

Code Snippet 3: LSTM Architecture

python

```
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(30, 1)),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')
```

9.4 Training Loop

Code Snippet 4: Model Training

python

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=100,
    batch_size=32,
    callbacks=[
        EarlyStopping(monitor='val_loss', patience=10,
                      restore_best_weights=True),
        ModelCheckpoint('best_model.h5', save_best_only=True)
    ],
    verbose=1
)
```

9.5 Evaluation

Code Snippet 5: Calculate Metrics

python

```python
# Predictions
predictions = model.predict(X_test)
predictions_inv = scaler.inverse_transform(predictions)
y_test_inv = scaler.inverse_transform(y_test)

# Metrics
mse = mean_squared_error(y_test_inv, predictions_inv)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test_inv, predictions_inv)
r2 = r2_score(y_test_inv, predictions_inv)

print(f"RMSE: {rmse:.4f} mm")
print(f"MAE: {mae:.4f} mm")
print(f"R²: {r2:.4f}")
```